Task manager simulator

Author: Lenni Sigfridsson
Updated: 3.4.2026

1. Overview

This application is created for simulating virtual task/process management with limited resources and queuing. It is created with the PySide6 Qt framework. The application includes a GUI and QThreads for concurrency. With the application, simulating doing different actions with tasks becomes easy because of clear structure and easy-to-understand functionality.

The application can be used for process simulation by developers, testers and end-users alike since its GUI and purpose are clear. With the application users can simulate tasks of varying (random) CPU and memory requirements. It includes a log which shows started and completed tasks, CPU and memory usage meters, and a measurement of the ETA for the currently running tasks. Users cannot, however, prioritize specific tasks or sort the tasks in a desired order. Task history is not persistent and does not include the CPU and memory requirements of each task.

2. Technologies

The application is developed with Python 3.13.9 and PySide6 version 6.10.0. Many different QtWidgets, QtCore and QtGui features are used in the application. These are listed below. In addition to those, the random and time modules are used.

Used PySide6 components
PySide6.QtWidgets
QApplication, QWidget, QMainWindow, QGridLayout, QVBoxLayout, QScrollArea, QLabel, QPushButton, QTextEdit, QProgressBar
PySide6.QtCore
Qt, QObject, QThread, Signal, Slot
PySide6.QtGui
QTextCursor

Table 1: Used PySide6 components

The usage of Qt framework components makes the UI very fast and responsive. More information about these can be found in the Qt docs:
https://doc.qt.io/qtforpython-6.10/PySide6/QtWidgets/

2.1. Purposes

There are brief explanations of the used components and what they're used for listed below. QApplication is the application that can be run and the GUI is built on that. QMainWindow is the main window that holds the GUI and is shown on screen. The GUI is built with QWidgets (which are, in fact QObjects) that hold other features, such as layouts (QGridLayout, QVBoxLayout), inside. Inside the layouts, widgets are placed and the GUI is built like this. The widgets are rendered on screen and some, like the QPushButton, are interactable.
The QTextCursor included in the QtGui is used to handle cursor location in a QTextEdit widget.

In addition to the GUI elements, the threading and signaling is also handled by Qt. Whenever a task is run, it is started as a QObject inside a QThread. The tasks are created in a Worker class, which creates Worker objects which inherit the QObject. All of the components can send and pass Signals to each other and there is a Slot in every Worker object that holds the functionality of the object. Signals are passed on from the Worker objects to display information on the GUI widgets.

3. Architecture

3.1. General remarks about architecture

The application's architecture consists of the GUI and classes that handle creation of said GUI and other parts of the application. Much of the functionality is bound to the GUI so many GUI related classes also handle some other functionality. In addition to those, there is the Worker class that handles creation of workers that are bound to threads. The resource management is also done separately from the GUI.

In hindsight, it could be smarter to separate tasks from task widgets. This is not done since it would further add complexity in, for example, transmitting signals from the workers to the GUI by adding a layer in between. A separate task manager could also be included. In this application it is included in the resource manager.

3.2. Class structure

The MainWindow class is the main class of the GUI. It extends Qt's QMainWindow and is where all the QWidgets are set up in. A QWidget is often put in the MainWindow. In this application there is a QWidget called MainUI which holds the other graphical elements inside. There is a layout in said MainUI that holds many widgets of the QtWidgets module. It also holds the TaskList created by the TaskList class which extends QWidget as well. The TaskList holds TaskWidget objects inside. These also extend QWidget. This is the way graphical elements are implemented in this application. The TaskList is placed in a QScrollArea in the MainUI and the TaskWidget objects are created inside.

In addition to these GUI-defining classes, there are the ResourceManager class and Worker class. In ResourceManager tasks and available CPU and memory resources are managed. It extends QObject and is created in the TaskList object since it is the only GUI widget that needs to know about managing tasks. The Worker also extends the QObject class and is responsible for being the template for the worker objects created in TaskWidget.

PySide6.QtCore's Signals are used to connect information between the classes and objects. Properties or values can be emitted and received using these Signals without having to import the entire object to all of the classes that use the value. These are created in many classes since they are emitted and re-emitted according to hierarchy. Some are created in the final 'backend' class, WorkerSignals which extends QObject. These signals are used by the Worker class. There is also a Styles class for having styles in one place. A style class is not necessary but it clarifies some things (like importing) so it is handy to have.
Below is a table of the classes and their relations:

Class	Extends	Created in	Purpose
MainWindow	QMainWindow	main.py upon GUI creation	Create base for GUI
MainUI	QWidget	MainWindow	Create widgets and a base for them
TaskList	QWidget	MainUI	Create a list of TaskWidgets
TaskWidget	QWidget	TaskList	Display a task and create the visual properties of a task
ResourceManager	QObject	TaskList	Handle resource and task management
Worker	QObject	TaskWidget	Create the properties of a task
WorkerSignals	QObject	Worker	Create signals to be sent by workers
Styles	-	All styled objects	Create styles

Table 2: Class inheritance and purposes

3.3. Control flows

The purpose of the application is to handle tasks and resources. All of the parts around this functionality are there to support it or the user's ability to accomplish this task. Supporting this task requires seamless and safe cooperation and communication between the modules and threads of the application. This is handled by PySide6.QtCore's Signals. These are quite difficult and unintuitive to use which is a challenge of this framework. They are required to handle communicating properties and variables between objects.

All of the signals are initially created in the WorkerSignals class. The Worker class uses these signals to emit different states of running in the run() method which is also defined as the @Slot(). For example, if a task is running in the worker, a progress and CPU and memory requirements signals are emitted to be used by other objects. These are then used or re-emitted by the TaskWidget objects and similarly, the TaskList objects re-emit these signals to either the ResourceManager to handle CPU and memory usage or to the MainUI where they are used to display different values. The ResourceManager also sends signals back to signal the current usage of the CPU and memory. This really shows the complexity and unintuitivity of the signals. The developer needs to use the signals to transmit data which needs to be altered via other objects. They need to be emitted and connected every time.

This all is to connect values and properties between objects. Of course, parameters are also used. Values of resource usage are passed on to each worker. These are then set as properties of the workers. Task names are defined as Task 1, Task 2, …, Task n. They are also passed on to the tasks upon creation.

Of course a clear task and event flow are also required for the application to be intuitive. Users can press buttons in the GUI. These buttons are QPushButtons which can be connected to an action. Initially, there are three buttons to press. These are the 'Add Task' button, the 'Clear log' button and the 'Exit' button. The Exit button closes the application, the Clear log button empties the log window and the Add task button adds a new task as a 'TaskWidget'. The added task is placed on the right side of the window. Multiple tasks can be added and if they don't fit in the window, the right side becomes scrollable and the user can still see all of the tasks.

The tasks have more buttons for controlling the task. The task can be paused and resumed which pauses the progress and time remaining. Canceling the task stops the task and releases the CPU and memory capacity it was using. After completion or cancelling, the tasks are still visible on the right side of the window. They are marked as ready or cancelled. They can be deleted by pressing the delete button which appears when the tasks are finished (one way or another). When the tasks are still running, they display either running or paused. Upon creation, the tasks are given a name that is displayed to separate tasks. The name is displayed in the log message followed by started or finished depending on the action.

3.4. Other structural solutions

Data in the application is passed on between objects as signals. It is then passed on to methods that need it by connecting signals to a method.
Example:

```
self.tasklist.cpu.connect(self.update_cpu_usage)
```

This connects the CPU usage signal passed by a TaskList object to the MainUI's method that updates the displayed value of CPU usage as follows:

```
def update_cpu_usage(self, usage):
        self.cpu_usage.setValue(usage)
```

This is the ideal case of the functionality. It happens if there is enough CPU (and memory) capacity to run the specified tasks. This is controlled in the ResourceManager. They are controlled separately because they are different properties of the task.
A method for checking the capacity is applied to the task that is trying to start. The task is then started if there's capacity and placed in queue if there is not enough capacity. The ResourceManager checks the entire queue for tasks that could run. There is not a specified order for the tasks to ensure efficient usage of resources. Before tasks are run. They are first placed in the queue and then run immediately if there's enough capacity.

4. Implementation Details

All of the files of this project are in a single directory. The files are listed below with short explanations:

main.py	Entry point
frontend.py	Houses the MainWindow and MainUI classes
tasklist.py	Houses the TaskList class
resources.py	Houses the ResourceManager class
tasks.py	Houses the TaskWidget class
workers.py	Houses the Worker and WorkerSignals classes
styles.py	Houses the Styles class

Table 3: File list with explanations

Application is created in main.py. The MainWindow and QApplication are imported there. Frontend is created in frontend.py, tasklist.py, tasks.py in combination with styles.py. 'Backend' functionalities are created in resources.py and workers.py.
frontend.py imports several QtWidgets and Qt and QTextCursor from QtCore and QtGui respectively. All of these are in the PySide6 package. These are then used to create widgets

and layouts (QtWidgets), setting alignments in layouts (by Qt) and moving the cursor in the log window (by QTextCursor). TaskList and Styles classes are also imported to create a TaskList object and setting styles (in MainUI).
tasklist.py imports QWidget and QVBoxLayout from the same QtWidgets package. They are used in the TaskList class creation (extension) and setting its layout.. Also, Signal is imported from QtCore to relay information to MainUI. TaskWidget and ResourceManager are also imported to create objects in this file.
resources.py imports the QObject to extend that and Signal to relay information back to the UI. It also handles some label editing (when setting tasks to running or queued) so it needs to import Styles to set styles.
tasks.py imports several QtWidgets to create graphical elements. It needs the QtCore's QThread to create threads for the workers to be run simultaneously. This is essential for the tasks to run and keep the UI responsible at the same time (in addition to the tasks being able to simultaneously). Signals are emitted so they also need to be imported. Workers are created (started) here so they need to be imported. The workers are started with random resource requirements so random is also imported. UI elements need to be styled so Styles is also imported.
workers.py imports QObject, Signal and Slot from QtCore. The class is created as a QObject and signals are emitted. Also the run() method acts as a @Slot() for the signals. The ETA of the tasks is calculated at this level so time is also imported.
styles.py doesn't import anything but is imported by the UI and ResourceManager classes.

The application is thread safe since the workers run independent of the UI. They also work flawlessly, being able to be run as resources allow it. The threads are objects extending the QThread. These can be killed when the app is exited. This is done as follows:

```
def exit_app(self):
        self.tasklist.exit_press()
        QApplication.quit()
```

and:

```
def exit_press(self):
        try:
            self.task.destroy_workers()
        except:
            pass
```

The chance for error is low here and the application exits without problems whether tasks are still running or not.

5. Testing and Improvements

Testing of the application has been completed as manual testing. Error handling has been added to spots where errors could occur. Testing was done mostly by printing values and sometimes objects to console. The state of the application was followed via console prints as well during testing. The most common errors that could occur are handled with Python's try-except structures with the workers and signals being the most error-prone. The workers' run loop concludes with a finishing confirmation and exceptions are added to handle possible errors in execution. These are evident in the GUI without the need to check a separate console.

There are some known limitations within the application. The console can be written in which could be a confusing property. Users can write notes regarding some task if they so desire. The console only shows the last 50 lines so their notes might disappear, though. Also, when a task is paused, the ETA calculator might not find the correct ETA value quickly or not at all. Sometimes the CPU value doesn't update/reset to 0 if a task is repeatedly paused and resumed. The app has crashed during development a few times without any error messages so there might be hidden errors. The newest version should be stable. It didn't crash during testing.

Planned improvements in the future include (in addition to bug fixes) improved scheduling and data persistence (in a text file, for example). The persistent data could hold the name and CPU and memory usage of the task. Prioritization of tasks could also be added in addition to reordering the tasks in the visible task list. Refactoring the code into more classes and a clear frontend/backend separation could also be considered in the future.

6. Gallery

Here are some screenshots of relevant functionalities/usage situations

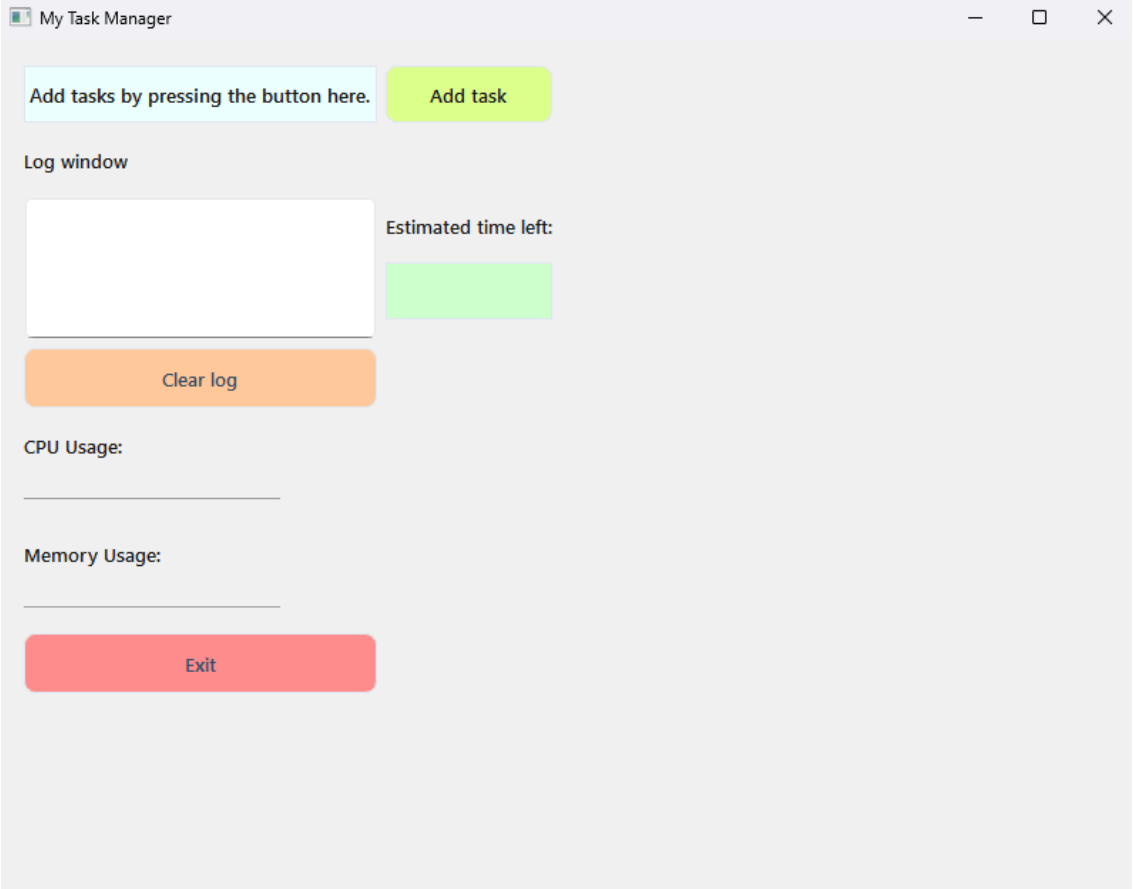

Figure 1: Initial view of the application (with an empty task list)
Buttons for adding a task, clearing the log and exiting the application
Labels for information and headers for log window, CPU usage, memory usage and ETA
Log window and the usage meters

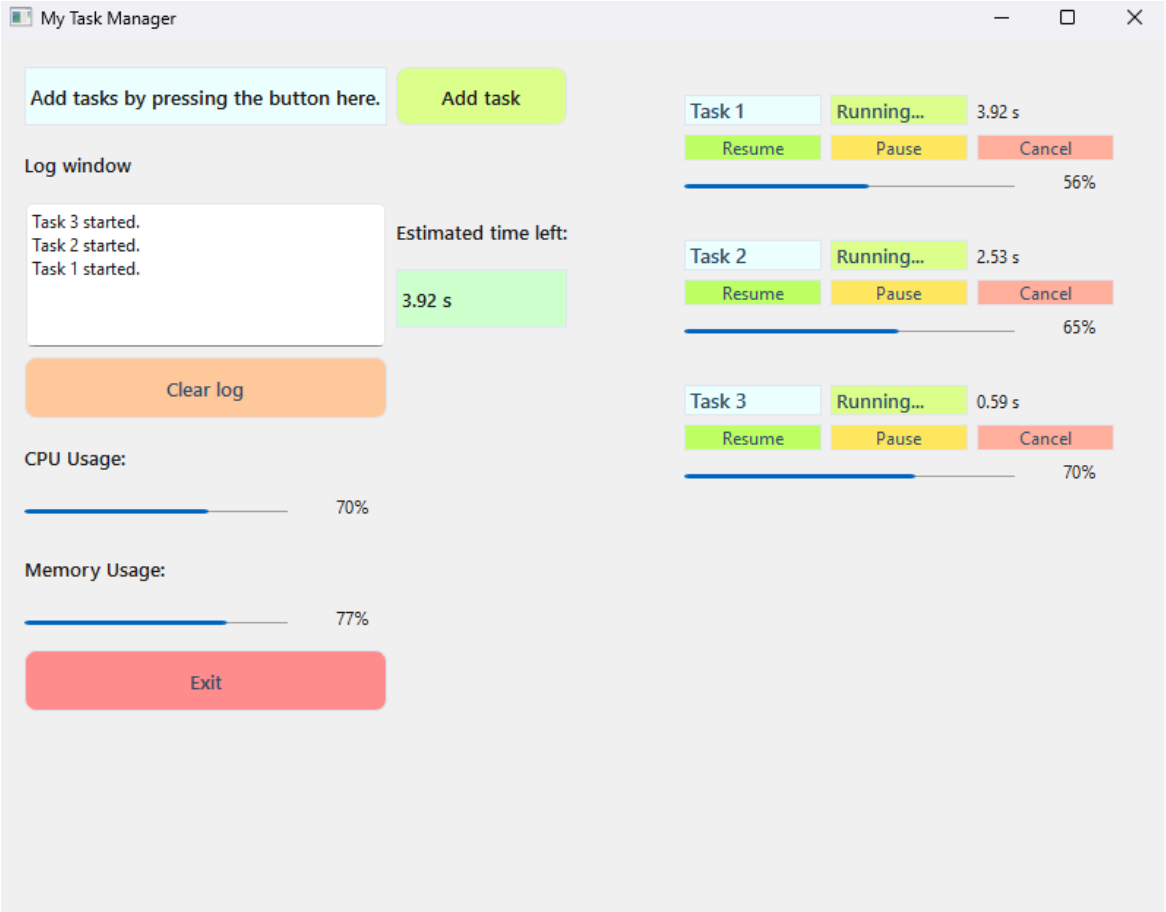

Figure 2: A few tasks are running. Tasks have their own labels and buttons and progress bars.
Labels for task name and status and ETA
Buttons for resuming/pausing and canceling.
CPU and Memory usage also shown at 70% and 77% respectively.
Log also shows start messages for tasks.

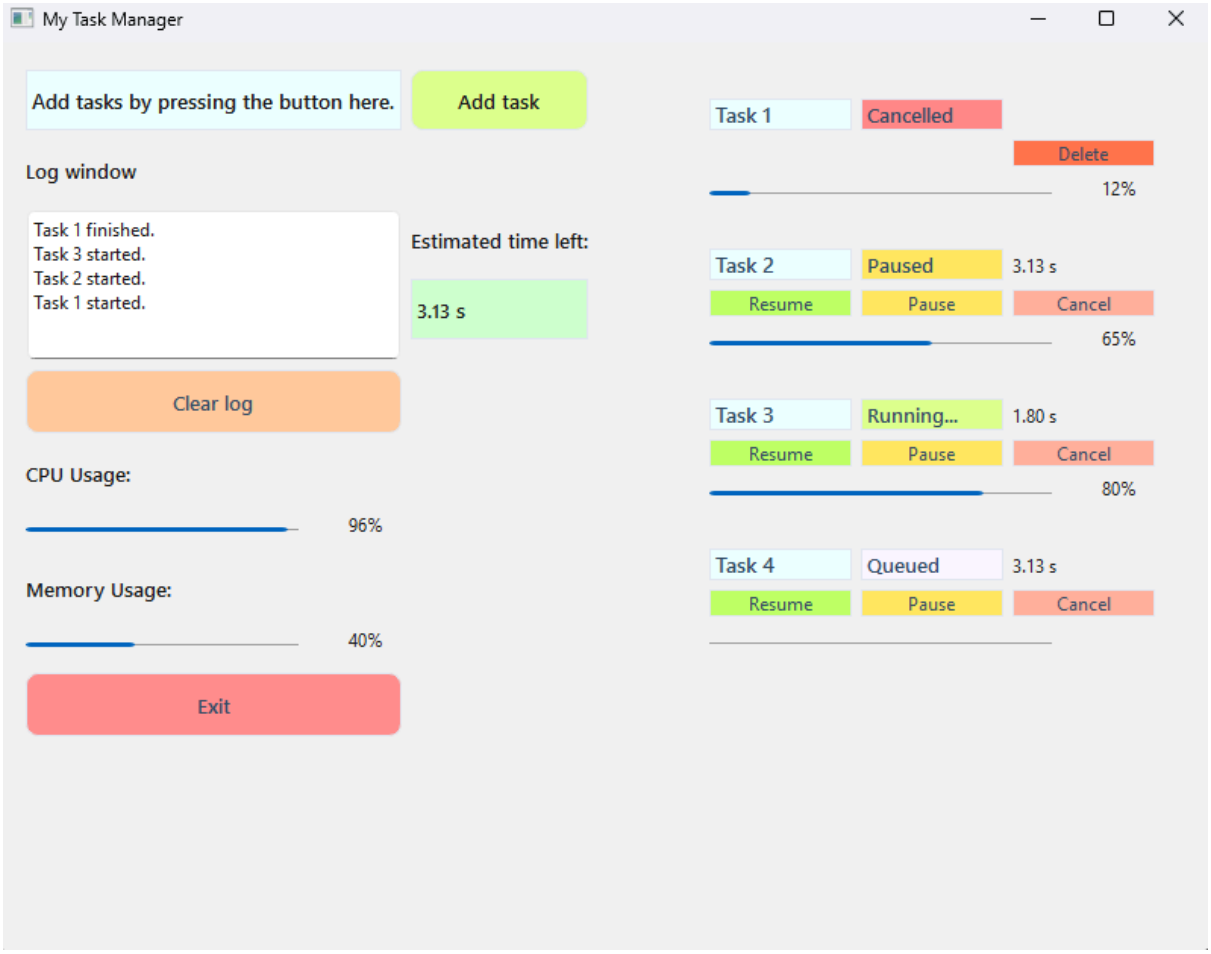

Figure 3: More tasks and statuses shown.
More task statuses are shown. One is cancelled, one is paused, one is running, and one is queued since there is not enough CPU capacity for running it.
The delete button is also visible for the cancelled task.

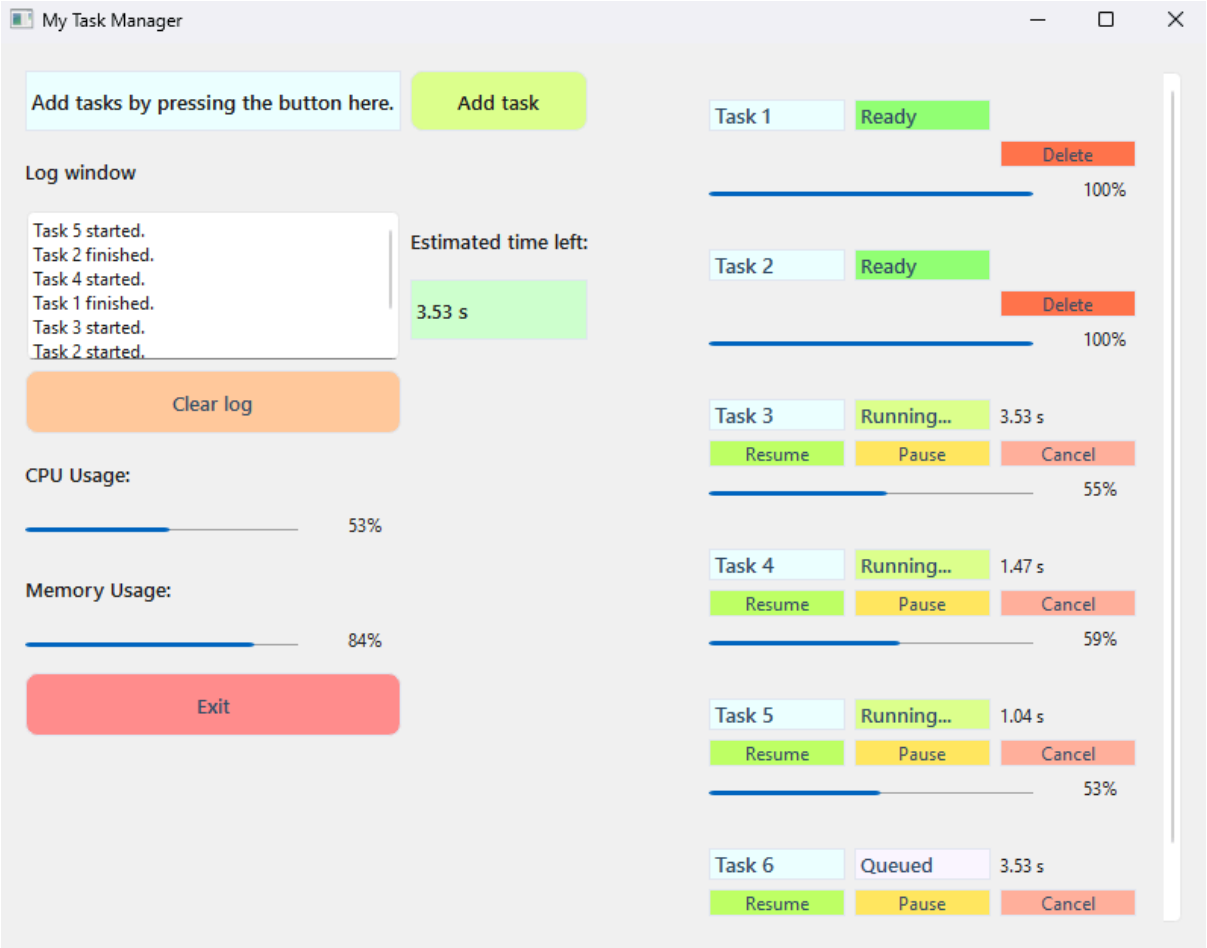

Figure 4: A view with the scrollable task list visible (too many to fit on screen)
A few tasks are ready and a few are running/queued. More tasks can be shown if the scroll area is scrolled down.

7. Usage of AI

AI was used in polishing the project idea and the application structure. It was also used to debug code and keep track of progress/features. In some parts it was used to suggest what to do next. Some implementation ideas were created by AI as well. No code was created solely by prompting with AI except for a few bug fixes.